

Кафедра
статистики и
кибернетики
*Department of statistics and
cybernetics*

Лекция №2

«Асимптотическая сложность. Алгоритмы сортировки. Рекурсия»

Лектор: Вадим Владимирович Демичев
доцент, кандидат экономических наук

Москва, 2026

Алгоритмы NP-класса:

- Класс задач P — это набор таких задач, для которых мы знаем алгоритм, работающий за полиномиальное время.
- Некоторые проблемы не решаются за разумное время.
- Класс NP определяется как задачи, для которых мы можем лишь проверить некоторое решение за полиномиальное время, но нам неизвестен алгоритм, который мог бы решить задачу настолько же быстро.

Правила:

- 1. Если для математической функции f алгоритму необходимо выполнить определенные действия $f(N)$ раз, то для этого ему понадобится сделать $O(f(N))$ шагов.
- 2. Если алгоритм выполняет одну операцию, состоящую из $O(f(N))$ шагов, а затем вторую операцию, включающую $O(g(N))$ шагов, то общая производительность алгоритма для функции f и g составит $O(f(N) + g(N))$

Правила-продолжение:

- 3. Если алгоритму необходимо сделать $O(f(N) + g(N))$ шагов и область значений N функции $f(N)$ больше, чем у $g(N)$, то асимптотическую сложность можно упростить до выражения $O(f(N))$
- 4. Если алгоритму внутри каждого шага $O(f(N))$ одной операции приходится выполнять еще $O(g(N))$ шагов другой операции, то общая производительность алгоритма составит $O(f(N) * g(N))$
- 5. Постоянными множителями (константами) можно пренебречь. Если C является константой, то $O(C * f(N))$ или $O(f(N * C))$ можно записать как $O(f(N))$

Пузырьковая сортировка:

- Сортировка простыми обменами, сортировка пузырьком (англ. bubble sort) — простой алгоритм сортировки.
- Алгоритм состоит из повторяющихся проходов по сортируемому массиву.
- За каждый проход элементы последовательно сравниваются попарно и, если порядок в паре неверный, выполняется обмен элементов. Проходы по массиву повторяются $N-1$ раз или до тех пор, пока на очередном проходе не окажется, что обмены больше не нужны, что означает — массив отсортирован.

Худшее время: $O(n^2)$
Среднее время: $O(n^2)$
Лучшее время: $O(n)$

Затраты памяти: $O(1)$

Пример

index	0	1	2	3	4	5	6	7	8	9
	20	44	93	31	17	54	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	93	31	17	54	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	93	31	17	54	55	65	77	26

Пример - продолжение:

index	0	1	2	3	4	5	6	7	8	9
	20	44	93	31	17	54	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	93	31	17	54	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	93	17	54	55	65	77	26

Пример - продолжение:

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	93	17	54	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	93	17	54	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	93	54	55	65	77	26

Пример - продолжение:

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	93	54	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	93	54	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	93	55	65	77	26

Пример - продолжение:

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	93	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	93	55	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	93	65	77	26

Пример - продолжение:

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	93	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	93	65	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	65	93	77	26

Пример - продолжение:

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	65	93	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	65	93	77	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	65	77	93	26

Пример - продолжение:

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	65	77	93	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	65	77	93	26

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	65	77	26	93

Пример - продолжение:

index	0	1	2	3	4	5	6	7	8	9
	20	44	31	17	54	55	65	77	26	93

index	0	1	2	3	4	5	6	7	8	9
	17	20	26	31	44	54	55	65	77	93

Реализация:

[9]
✓ 0 сек.

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n-1):  
        for j in range(n-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j],arr[j+1] = arr[j+1], arr[j]  
    return arr
```

[10]
✓ 0 сек.

```
a = [1,0,5,12,3,1,2,6,4,3]  
a_sort = bubble_sort(a)
```

[12]
✓ 0 сек.



a_sort



... [0, 1, 1, 2, 3, 3, 4, 5, 6, 12]

Сортировка выбором:



СЧЕТЧИК ВОС-ПРОИЗВЕДЕНИЙ

RADIOHEAD	156
KISHORE KUMAR	141
THE BLACK KEYS	35
NEUTRAL MILK HOTEL	94
BECK	88
THE STROKES	61
WILCO	111

Первый шаг:

~ ♪ ~	СЧЕТЧИК ВОС- ПРОИЗВЕДЕНИЙ	→	СПИСОК	СЧЕТЧИК ВОС- ПРОИЗВЕДЕНИЙ
RADIOHEAD	156		RADIOHEAD	156
KISHORE KUMAR	141			
THE BLACK KEYS	35			
NEUTRAL MILK HOTEL	94			
BECK	88			
THE STROKES	61			
WILCO	111			

1. У RADIOHEAD БОЛЬШЕ ВСЕГО ВОСПРОИЗВЕДЕНИЙ...

2. ДОБАВЛЯЕМ RADIOHEAD В НОВЫЙ СПИСОК

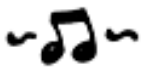
Второй шаг:

~ ♪ ~	СЧЕТЧИК ВОСПРОИЗВЕДЕНИЙ	→	♪ СПИСОК ♪	СЧЕТЧИК ВОСПРОИЗВЕДЕНИЙ
			RADIOHEAD	156
KISHORE KUMAR	141		KISHORE KUMAR	141
THE BLACK KEYS	35			
NEUTRAL MILK HOTEL	94			
BECK	88			
THE STROKES	61			
WILCO	111			

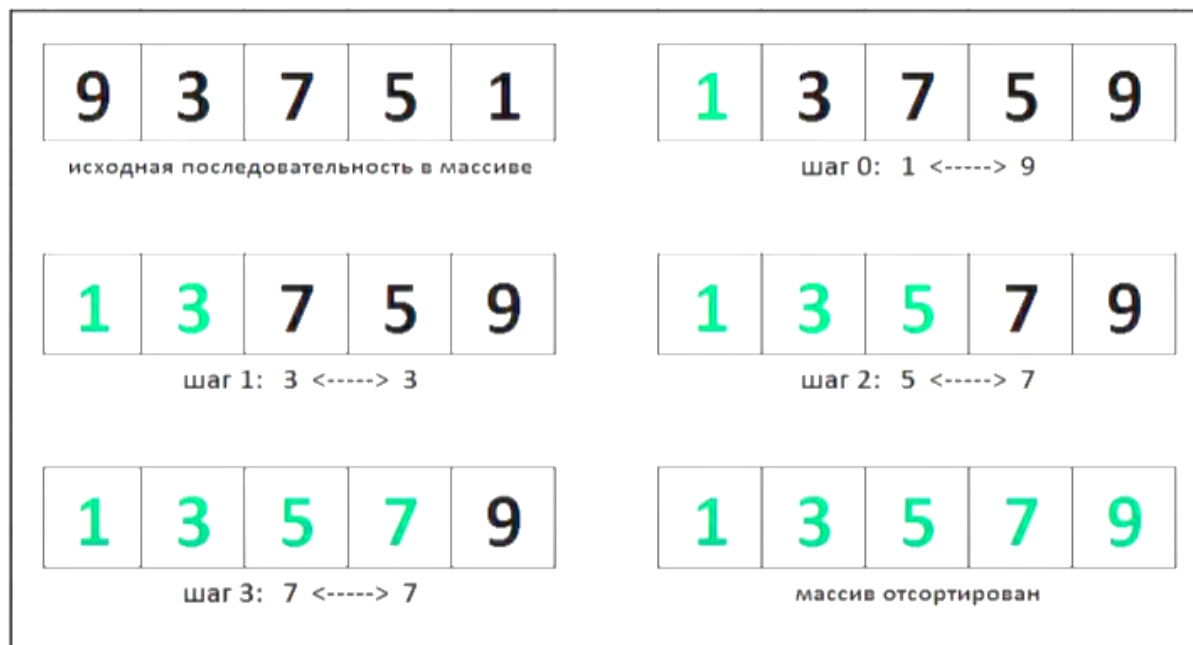
1. СЛЕДУЮЩИЙ ИСПОЛНИТЕЛЬ ПО КОЛИЧЕСТВУ ВОСПРОИЗВЕДЕНИЙ - KISHORE KUMAR

2. ПОЭТОМУ ОН СЛЕДУЮЩИМ ДОБАВЛЯЕТСЯ В НОВЫЙ СПИСОК

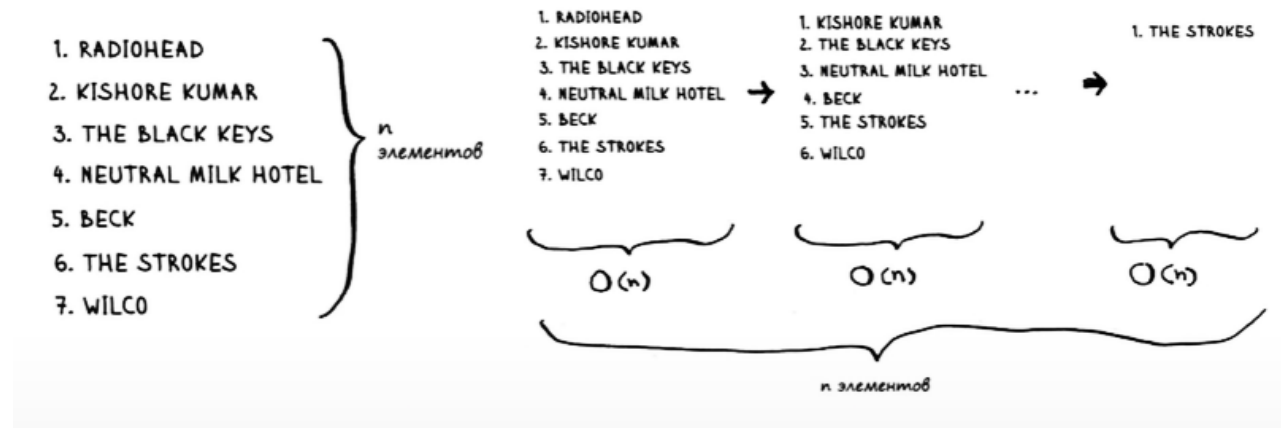
Результат:

	СЧЕТЧИК ВОС- ПРОИЗВЕДЕНИЯ
RADIOHEAD	156
KISHORE KUMAR	141
WILCO	111
NEUTRAL MILK HOTEL	94
BECK	88
THE STROKES	61
THE BLACK KEYS	35

Еще один пример с числами:



Сложность $n*n$:



Поиск наименьшего значения:

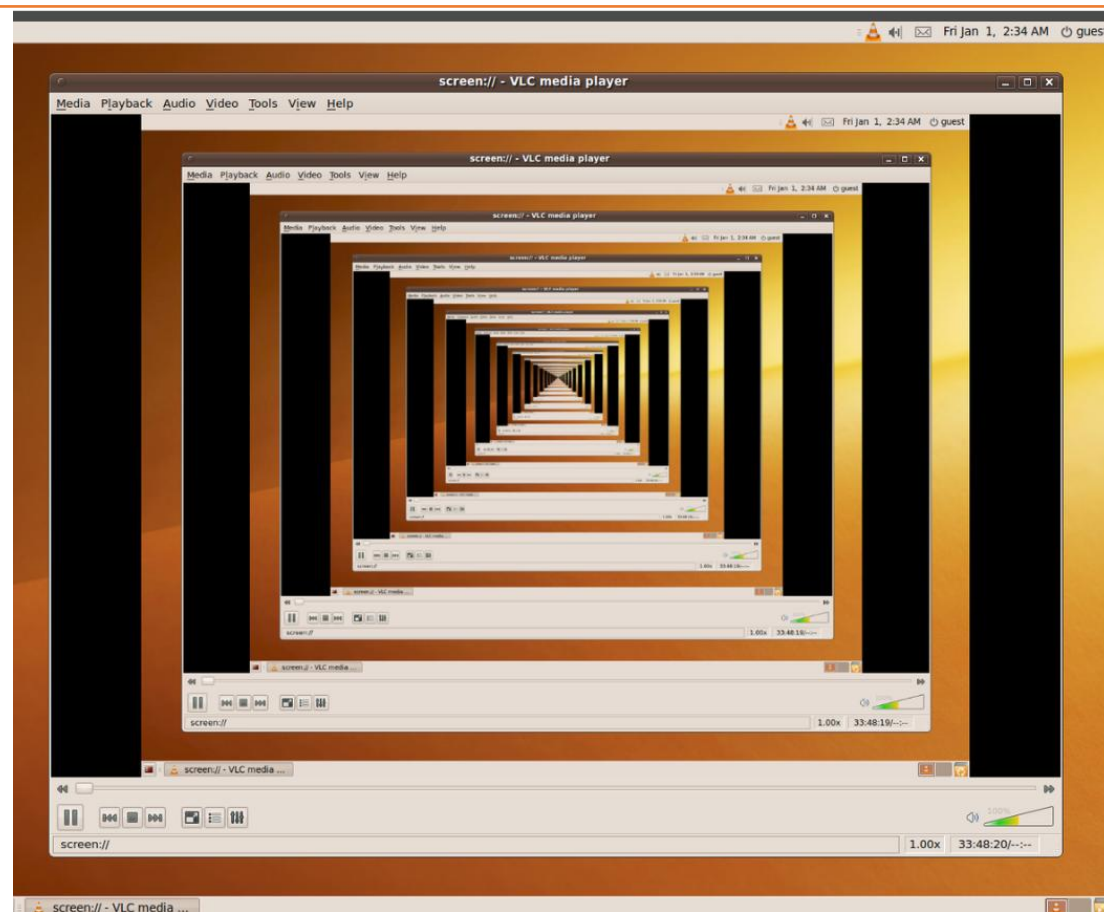
```
1  def find_Smallest(arr):
2      smallest = arr[0] #для хранения наименьшего значения
3      smallest_index = 0 #для хранения индекса наименьшего значения
4      for i in range(1, len(arr)):
5          if arr[i] < smallest:
6              smallest = arr[i]
7              smallest_index = i
8      return smallest_index
9
```

Реализация сортировки выбором:

```
1  #1 часть
2  def find_smallest(arr):
3      smallest = arr[0] #Для хранения наименьшего значения
4      smallest_index = 0 #Для хранения индекса наименьшего значения
5      for i in range(1, len(arr)):
6          if arr[i] < smallest:
7              smallest = arr[i]
8              smallest_index = i
9      return smallest_index
10
11 #2 часть
12 def selectionSort(arr): #Сортирует массив
13     newArr = []
14     for i in range(len(arr)):
15         smallest = find_smallest(arr) '''Находит наименьший элемент в массиве
16         и добавляет его в новый массив'''
17
18         newArr.append(arr.pop(smallest))
19     return newArr
20
21 print(selectionSort([5,3,6,2,10]))
22
23
```

```
Projects')
[2, 3, 5, 6, 10]
```

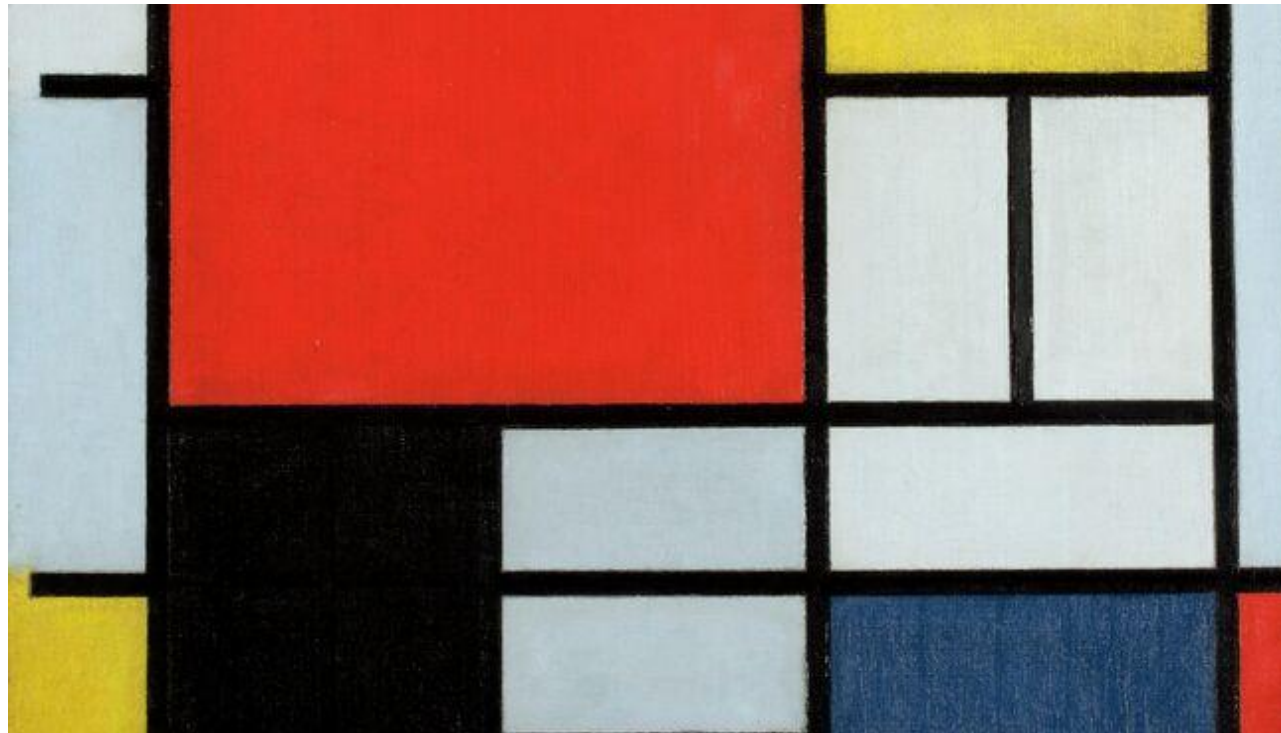
Рекурсия



Рекурсия. Эффект Дросте:



Рекурсия в искусстве



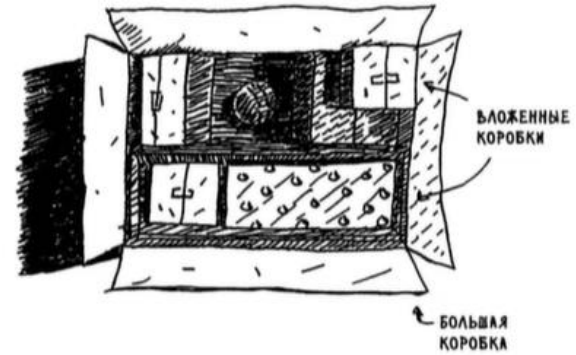
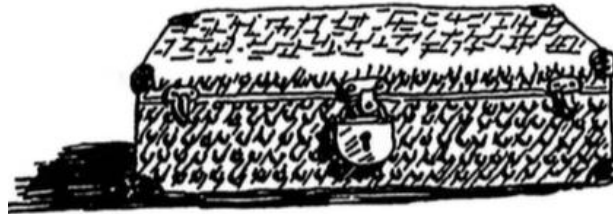
С помощью приведенной рекурсивной функции на прямоугольном холсте можно создавать картины в стиле нидерландского живописца **Пита Мондриана (Piet Mondrian, 1872–1944)**. На его картинах прямоугольный холст разделен на последовательно уменьшающиеся прямоугольники. Следуя данной процедуре, попытайтесь самостоятельно создать картины, аналогичные изображенной на рисунке. Начните с прямоугольника, представляющего весь холст, на котором вы работаете.

def Mondrian(Прямоугольник):

if (размер прямоугольника слишком велик для вашего художественного восприятия):

разделите прямоугольник на два меньших прямоугольника
примените функцию **Mondrian** к первому из двух меньших
прямоугольников и закрасьте его некоторым цветом
примените функцию **Mondrian** ко второму из двух меньших
прямоугольников и закрасьте его другим цветом

Итеративный подход:



Итеративный подход-продолжение:



Рекурсивный подход:



Рекурсивная функция:

```
def countdown(i):  
    print (i)  
    countdown(i-1)
```

```
countdown(10)
```



Рекурсивный подход - продолжение:

```
def countdown(i):  
    print (i)  
    countdown(i-1)  
  
countdown(10)
```

10
9
8
7
6
5
4
3
2
1
0
-1
-2
-3
-4
-5
-6
-7
-8
-9
-10
-11
-12
-13
-14
-15
-16
-17
-18
-19
-20
-21

Рекурсивная функция:

```
def countdown(i):  
    print i  
    if i <= 0:      ← ..... Базовый случай  
        return  
    else:           ← ..... Рекурсивный случай  
        countdow n(i-1)
```



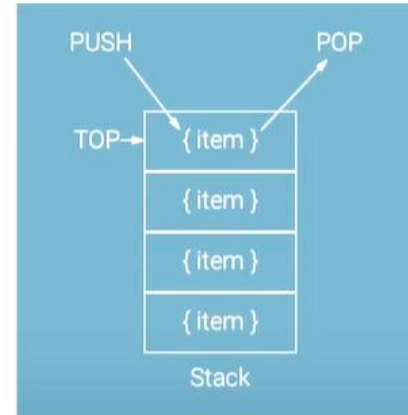
Рекурсивная функция:

```
def countdown(i):  
    print i  
    if i <= 0:  ←..... Базовый случай  
        return  
    else:      ←..... Рекурсивный случай  
        countdow n(i-1)
```

```
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
0  
> []
```

Стек

- Стек — список элементов, который может быть изменен лишь с одной стороны, называемой вершиной стека.
- В рамках структур данных это называется LIFO-принципом (последним пришёл — первым ушёл).
- Стек поддерживает операции добавления (push) и удаления (pop) элементов на его вершине.



Стек



ЗАНЕСЕНИЕ
(НОВЫЙ ЭЛЕМЕНТ
ДОБАВЛЯЕТСЯ
НА ВЕРХ СТОПКИ)



ИЗВЛЕЧЕНИЕ
(ВЕРХНИЙ ЭЛЕМЕНТ
ВЫВОДИТСЯ ИЗ СТОПКИ
И ЧИТАЕТСЯ)

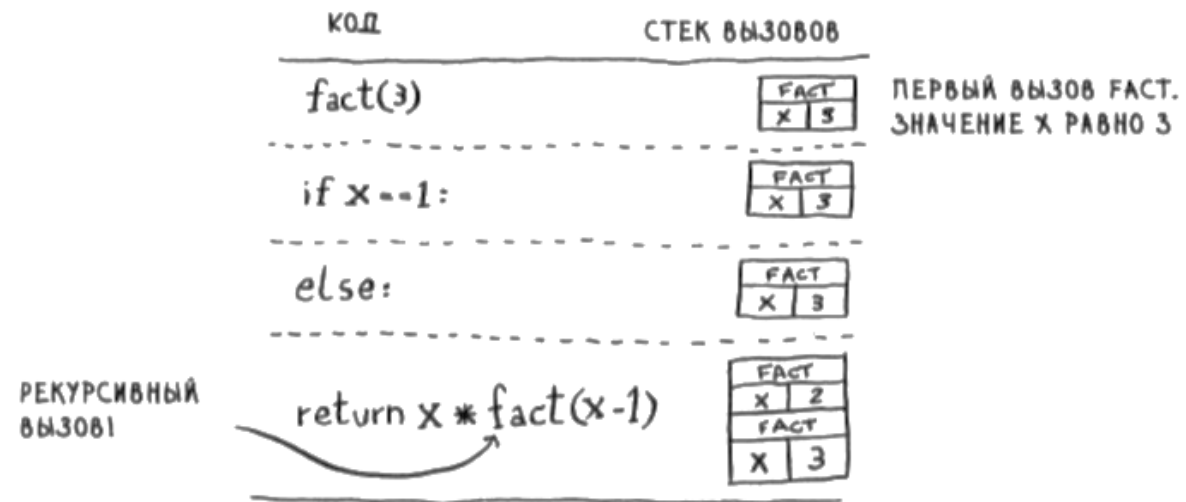
Рекурсия

- В программировании рекурсия связана с функциями - рекурсивные функции.
- Определение части функции через саму себя, то есть это функция, которая вызывает саму себя.
- Рекурсия это своего рода перебор.
- Все, что решается итеративно можно решить рекурсивно, то есть с использованием рекурсивной функции.
- Любой алгоритм, реализованный в рекурсивной форме, может быть переписан в итерационном виде и наоборот.

Рекурсивная функция

```
1 def fact(x):  
2     if x == 1:  
3         return 1  
4     else:  
5         return x*fact(x-1)  
6
```

Рекурсивная функция



Рекурсивная функция

СЕЙЧАС ТЕКУЩИМ
СТАЛ ВТОРОЙ ВЫЗОВ
ФАКТ. ЗНАЧЕНИЕ X
РАВНО 2

if $x == 1$:

else:

return $x * \text{fact}(x-1)$

ФАКТ	
x	2
ФАКТ	
x	3

← ВЕРХНИЙ ВЫЗОВ ФУНКЦИИ – ТОТ, КОТОРЫЙ В ДАННЫЙ МОМЕНТ ЯВЛЯЕТСЯ ТЕКУЩИМ

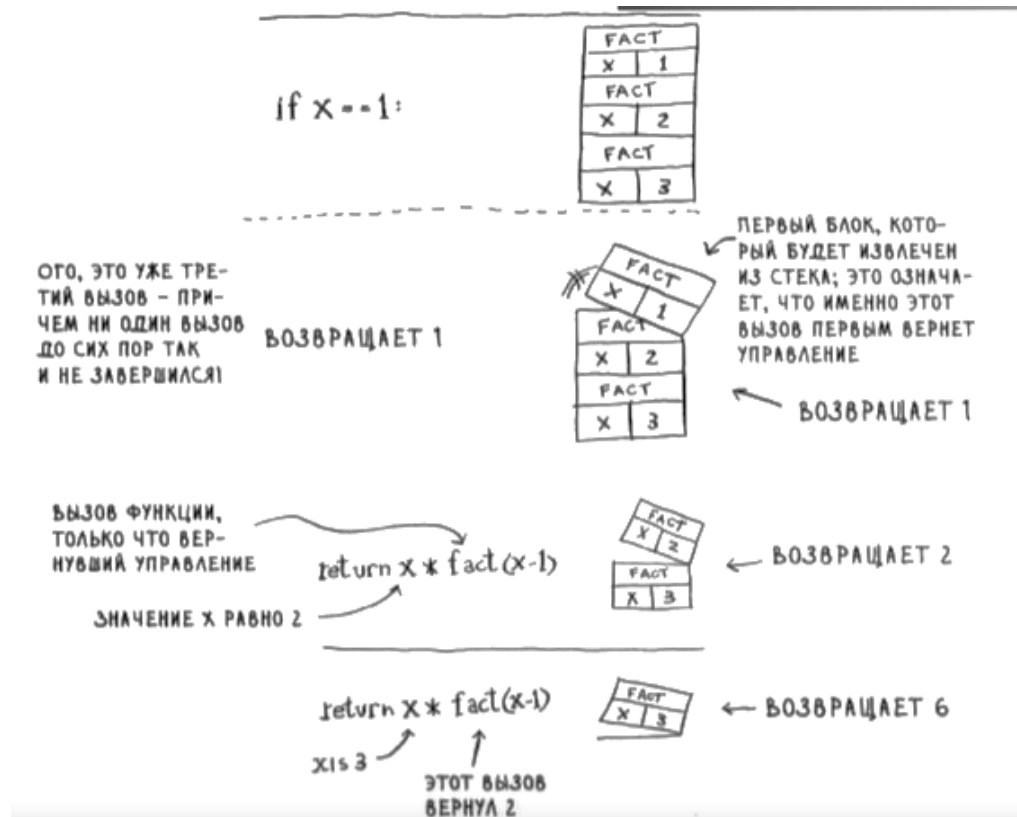
ФАКТ	
x	2
ФАКТ	
x	3

← В ОБОИХ ВЫЗОВАХ СУЩЕСТВУЕТ ПЕРЕМЕННАЯ С ИМЕНЕМ x, КОТОРАЯ ИМЕЕТ В ЭТИХ ВЫЗОВАХ РАЗНЫЕ ЗНАЧЕНИЯ

ФАКТ	
x	1
ФАКТ	
x	2
ФАКТ	
x	3

← ОБРАТИТЬСЯ К ЗНАЧЕНИЮ x ЭТОГО ВЫЗОВА ВНУТРИ ЭТОГО ВЫЗОВА НЕВОЗМОЖНО – И НАОБОРОТ

Рекурсивная функция



Что делает функция?

```
1  def funct(n):  
2      if n < 10:  
3          return n  
4      else:  
5          return n % 10 + funct(n//10)  
6  
7  print(funct(123))
```

Что делает функция?

```
1  def funct(k, n):  
2      if k < n:  
3          return str(k) + " " + str(funct(k+1, n))  
4      else:  
5          return n  
6  
7  print(funct(5, 8))  
8  
9
```

Ключевые тезисы

- Когда функция вызывает саму себя, это называется рекурсией.
- В каждой рекурсивной функции должно быть два случая: базовый и рекурсивный.
- Стек поддерживает две операции: занесение и извлечение элементов.
- Все вызовы функции сохраняются в стеке вызовов.
- Если стек вызовов станет очень большим, он займет слишком много памяти.

Вопросы

